

FOR LEADERSHIP

Spec-Driven Development

Shipping the right thing — predictably.

How we turn business requirements into working software — with humans approving every critical step. Especially important now that AI can build fast: we need to make sure that speed points in the right direction.

Four patterns that derail most software projects

These four patterns show up in nearly every team — and each one compounds the others.



Expensive rework

We build a feature, ship it, and then discover it wasn't what the business actually needed — and do it again. On average, 40–50% of total project effort goes to work that gets thrown away or redone.

Every rework cycle: wasted time, delayed roadmap, team morale hit.



Fuzzy requirements

"What to build" and "why it matters" live in Slack threads, meeting notes, and people's heads — not in one single, approved place. Everyone has a slightly different understanding.

Result: engineers build confidently in the wrong direction.



Silent scope creep

Work quietly expands mid-build. New ideas get added, edges cases multiply, but there's no formal decision point — so the timeline slips with no clear explanation for why.

Nobody is to blame, but everyone suffers the delay.



AI without guardrails

AI can now write code in hours that would have taken days. That's powerful — but speed in the wrong direction is more damaging, not less. AI amplifies the cost of unclear requirements.

Faster wrong = harder to recover.

The cost of the status quo is going up — not down

Three reasons this is urgent in 2025.



40–50% of rework

Industry research consistently finds that 40–50% of all software rework is traceable to unclear or misunderstood requirements at the start. This isn't a code quality problem — it's a communication problem.

→ *The fix is a better process before the first line of code, not better testing afterward.*



AI makes it 10× faster — and 10× riskier

A team using AI today can build in one week what used to take a month. That's a force multiplier — but only if the direction is right. Unclear requirements + AI speed = very expensive wrong turns, very fast.

→ *Speed amplifies both good direction and bad direction equally.*



No signed-off record

Right now, there is often no single moment where the business said "yes, build exactly this." When the wrong thing ships, there's no accountability — and no learning. The same patterns repeat next sprint.

→ *SDD creates that moment. And it creates the paper trail to learn from.*

Agree on **what & why** first.
Then build against an **approved spec** —
with **humans signing off** at every critical step.



Requirement-first

The spec is written in business language — WHAT and WHY only. No tech jargon. A business owner can read it, own it, and approve it without an engineer in the room.



Approval-gated

Three human checkpoints: approve the spec, approve the plan, say go to build. Work cannot advance past a gate without an explicit sign-off. No silent progress.

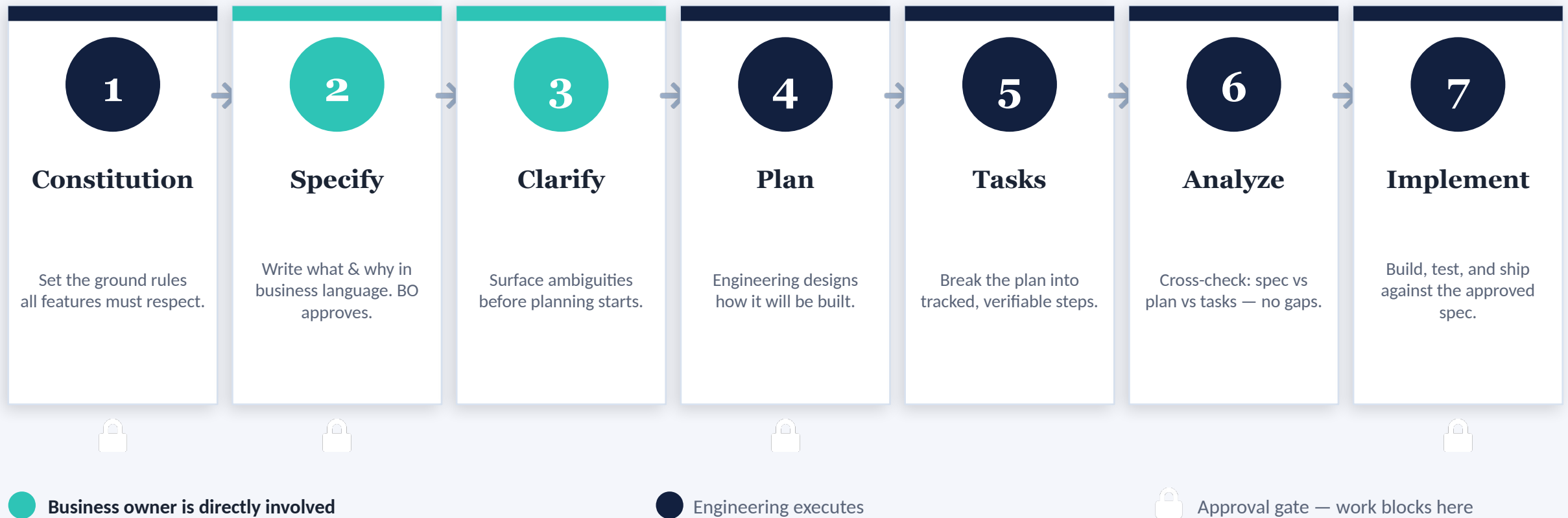


Fully traceable

Every task traces back to the spec. Every line of code traces back to a task. At any moment, you can answer "why was this built?" with a document.

Every feature follows the same 7-step path

Each step produces a written artifact. The next step cannot start until the previous one is complete.



THE STEPS YOU'RE DIRECTLY INVOLVED IN

Phases 1–3: where business strategy meets the process

These three steps happen before any engineering work begins. Your involvement here prevents every costly surprise later.

PHASE 1
Constitution

Before any feature begins, the team documents the principles that govern the entire product: design standards, technical constraints, compliance rules, what we will and won't do. This document is written once and applies to every feature that follows.

PRODUCES  **constitution.md**

OWNER PM + Tech Lead

Without this, every team member makes their own judgment calls. Inconsistency builds up silently across features.

PHASE 2
Specify

The business owner (or PM) writes — in plain English — what the feature should do, why it matters to the user, and how you'll know it's done. No technical language. This spec is the source of truth for everything that follows.

PRODUCES  **spec.md**

OWNER Business Owner / PM

 **APPROVAL GATE**
Engineering cannot start planning until this spec is formally approved. A vague spec produces a vague product.

PHASE 3
Clarify

Before engineering plans anything, the team reviews the spec together to surface any ambiguity or unstated assumption. Questions are answered, edge cases are added to the spec. This step typically takes one meeting.

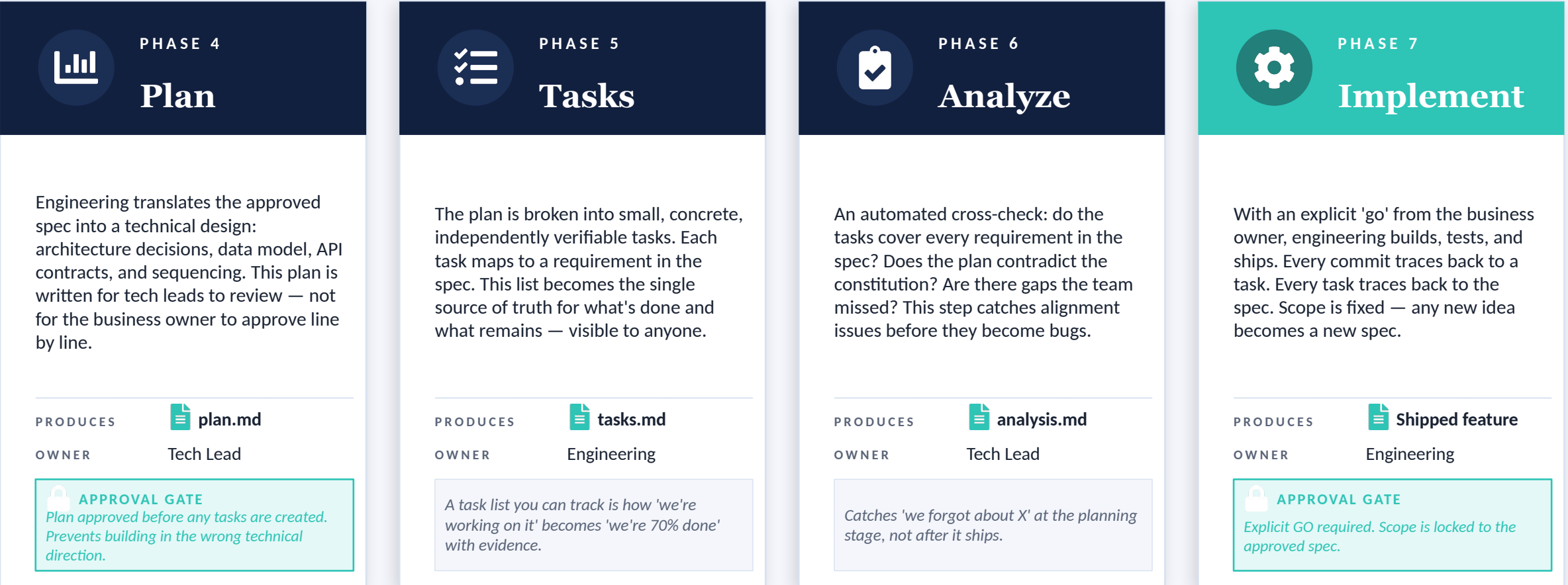
PRODUCES  **Updated spec.md**

OWNER PM + Engineering

Ambiguity caught here costs 30 minutes. Ambiguity discovered mid-build costs days and rework.

Phases 4–7: approved spec to shipped software

Once the spec is approved, engineering takes over. You remain a sign-off at gate 2 (plan) before any build starts.



Three approval gates keep humans in charge

Work cannot advance past a gate without an explicit sign-off. If the spec isn't right, nothing moves. If the plan isn't right, nothing is built.



GATE 1

Spec approved

Who approves: Business Owner / PM

The business owner reads the spec and confirms: "Yes, this is what we want and why we want it. I'm comfortable with engineering planning against this." No engineering work has started yet.

WHEN: After Specify + Clarify, before Plan

If the spec is wrong, this is the cheapest possible moment to fix it — before anyone has designed or built anything.



Without this gate, engineering makes assumptions about what you want. Some will be right. Some won't.



GATE 2

Plan approved

Who approves: Tech Lead (+ PM review)

Engineering presents how they'll build it: the architecture, the data design, the key risks. The tech lead signs off that the approach is sound, safe, and achievable within scope.

WHEN: After Plan, before Tasks are created

If the plan is architecturally unsound, this is the moment to redesign — before any code is written.



Without this gate, you may get code that technically works but is impossible to maintain or extend.



GATE 3

Go to build

Who approves: Business Owner

An explicit, documented "go" signal before the first line of code is written. The spec is locked. Any new idea that comes up during implementation becomes a new feature — it doesn't silently expand the current one.

WHEN: After Tasks + Analyze, before Implement

This is the moment where scope is frozen and accountability is established. You approved what gets built.



Without this gate, scope creeps, deadlines slip, and nobody is sure exactly what was agreed to.

Everyone owns the part they're best at

SDD doesn't ask the business owner to write code or engineers to define product strategy. Each person stays in their lane.



Business Owner / PM

Owns the WHAT and the WHY



Set the ground rules

Define the principles the product must follow.



Write & approve the spec

Plain English — no technical jargon required.



Answer clarifying questions

One meeting to remove any ambiguity from the spec.



Approve the plan & go

Two sign-offs before a single line of code is written.



Engineering Team

Owns the HOW



Translate spec into a plan

Architecture, data model, key technical decisions.



Break plan into tasks

Tracked, verifiable steps mapped to requirements.



Build against the spec

Scope is fixed; new ideas become new specs.



Flag conflicts early

If spec and plan don't align, raise it before building.

From ad-hoc to accountable

The same team, the same people — just a different process.

WITHOUT SDD



Requirements scattered in chats

No single source of truth. Everyone's working from a slightly different version.



"Is this done?" has no clear answer

Progress is invisible until it's finished — or late.



Scope grows without a decision

Features expand silently. Nobody approved the expansion.



AI writes code nobody spec'ed

The speed is there, but it's building against assumptions.



Rework discovered after shipping

Spec-Driven Development

The most expensive moment to find a problem.

WITH SDD



One approved spec — written down

Everybody reads the same document. Disagreements surface early, not late.



Status is visible at every step

tasks.md shows exactly what's done, in progress, and remaining.



Scope changes are explicit decisions

Any new idea becomes a new spec. The current build is locked.



AI builds against an approved spec

Speed applies to the right direction. No guessing.



Problems caught at the gate

The spec review is 30 minutes. A rework cycle is 2 weeks.

What this buys the business

Each benefit is a direct consequence of the process — not a side effect.

Predictability

Every feature follows the same path. Timelines are easier to estimate because the process is consistent. Surprises happen at the spec stage — not at the launch.

Same process → same predictability → better planning.

Less rework

Catching "wrong thing" at the 30-minute spec review costs almost nothing. Catching it after 2 weeks of build costs 2 weeks plus morale plus opportunity cost.

The spec review is the cheapest QA step in the entire process.

Full traceability

Any line of code can be traced back to a task. Any task can be traced back to a spec. Any spec was approved by a named person. You always have an answer for "why was this built?"

No more mystery features, no more forgotten context.

AI with guardrails

AI is now the most powerful accelerator in software development — but only when it's pointed in the right direction. SDD provides the approved spec that AI builds against, so speed doesn't become a liability.

AI multiplies your velocity. SDD makes sure that velocity is directed.

THE ASK

Let's pilot it on one feature

Pick one real upcoming feature and run it end-to-end through SDD. You'll see a spec written for you to read and approve, the approval gates in action, and a full paper trail of what was agreed to and why — before a single line of code is written. Low risk, real evidence.



Step 1: Choose a feature

Pick one real upcoming feature from the current roadmap — small or medium scope is best for a first run.



Step 2: Name the approver

Identify who will approve the spec (what/why) and give the go before the build. Usually the PM or product owner.



Step 3: Review the result

After implement, we walk you through the full paper trail: spec → plan → tasks → shipped, all traceable.